

Assignment 12

Advanced Programming

ARITRA TALUKDAR,CSB24034

5th May, 2026

Title

E-Commerce Order Processing System using SOLID Principles

Objective

To design and implement an extensible and maintainable E-commerce Order Processing System using Python, following SOLID design principles.

Introduction

This program simulates an E-commerce order processing system. It demonstrates clean software architecture by applying SOLID principles such as Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.

The system handles:

- Order creation
- Payment processing
- Notification sending
- Data storage
- Pricing strategies

Explanation of the Program

1. Data Model (Order Class)

The `Order` class is implemented using Python's `dataclass`. It stores all order-related information such as customer name, items, total amount, order type, and status.

It follows the **Single Responsibility Principle (SRP)** as it only holds data.

2. Payment Processing

An abstract class `PaymentProcessor` defines the method `process_payment()`.

Concrete implementations:

- `CreditCardPayment`
- `UPIPayment`
- `WalletPayment`

This follows:

- **Open-Closed Principle (OCP)**: New payment methods can be added without modifying existing code.
- **Interface Segregation Principle (ISP)**: Focused interface for payment only.

3. Notification System

The abstract class `NotificationSender` defines the method `send()`.

Implementations:

- `EmailNotification`
- `SMSNotification`
- `PushNotification`

Each class handles a specific notification channel, maintaining modularity and extensibility.

4. Storage System

The abstract class `OrderStorage` defines the method `save()`.

Concrete implementations:

- `DatabaseStorage` (simulated database insert)
- `FileStorage` (stores order in JSON file)

This demonstrates:

- **Dependency Inversion Principle (DIP)**
- Flexibility in choosing storage mechanisms

5. Pricing Strategy

The abstract class `OrderPricing` defines `final_amount()`.

Implementations:

- `RegularOrderPricing`
- `DiscountedOrderPricing`
- `PriorityOrderPricing`

This follows:

- **Open-Closed Principle (OCP)**
- **Liskov Substitution Principle (LSP)**

Each pricing strategy modifies the total amount independently.

6. Order Service

The `OrderService` class acts as the main controller of the system.

Responsibilities:

- Calculate final amount
- Process payment
- Save order
- Send notifications

It uses dependency injection, meaning it depends only on abstract classes, not concrete implementations.

7. Program Execution

The main function demonstrates four scenarios:

- Regular order with credit card payment
- Discounted order with UPI payment
- Priority order with wallet payment (sufficient balance)
- Failed order due to insufficient wallet balance

Source Code

```
# =====
#   E-COMMERCE ORDER PROCESSING SYSTEM
#   Designed with SOLID Principles in Python
# =====

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime
import json, uuid

# DATA MODEL   (Single responsibility: just hold data)

@dataclass
class Order:
    customer_name: str
    items: list
    total_amount: float
    order_type: str = "Regular"
    order_id: str = field(default_factory=lambda: str(uuid.uuid4())[:8])
    created_at: str = field(default_factory=lambda: datetime.now().isoformat())
    status: str = "Pending"

# PAYMENT INTERFACE   (ISP: small, focused interface)

class PaymentProcessor(ABC):
    @abstractmethod
    def process_payment(self, amount: float) -> bool:
        """Returns True if payment succeeds."""

# Concrete payment methods - OCP: add new ones without touching
# existing code
class CreditCardPayment(PaymentProcessor):
    def process_payment(self, amount: float) -> bool:
        print(f" [Credit Card] Charged Rs{amount:.2f} - Approved [OK]")
        return True

class UPIPayment(PaymentProcessor):
    def process_payment(self, amount: float) -> bool:
        print(f" [UPI] Payment of Rs{amount:.2f} via UPI - Success [OK]")
        return True
```

```

class WalletPayment(PaymentProcessor):
    def __init__(self, balance: float):
        self.balance = balance

    def process_payment(self, amount: float) -> bool:
        if self.balance >= amount:
            self.balance -= amount
            print(f" [Wallet] Debited Rs{amount:.2f} | Remaining
                : Rs{self.balance:.2f} [OK]")
            return True
        print(f" [Wallet] Insufficient balance [FAIL]")
        return False

# NOTIFICATION INTERFACE (ISP: separate from payment)

class NotificationSender(ABC):
    @abstractmethod
    def send(self, order: Order) -> None:
        """Send a notification for the given order."""

# OCP: new channel = new class, zero changes elsewhere
class EmailNotification(NotificationSender):
    def send(self, order: Order) -> None:
        print(f" [Email] Order #{order.order_id} confirmed for {
            order.customer_name}")

class SMSNotification(NotificationSender):
    def send(self, order: Order) -> None:
        print(f" [SMS] Hi {order.customer_name}! Your order #{
            order.order_id} is placed.")

class PushNotification(NotificationSender):
    def send(self, order: Order) -> None:
        print(f" [Push] Order #{order.order_id} placed
            successfully!")

# STORAGE INTERFACE (ISP + DIP)

class OrderStorage(ABC):
    @abstractmethod
    def save(self, order: Order) -> None:
        """Persist the order."""

class DatabaseStorage(OrderStorage):

```

```

    """Simulates a DB insert."""
    def save(self, order: Order) -> None:
        print(f"    [DB] INSERT order #{order.order_id} -> orders
              table right!")

class FileStorage(OrderStorage):
    """Appends order as JSON to a local file."""
    def __init__(self, filepath: str = "orders.json"):
        self.filepath = filepath

    def save(self, order: Order) -> None:
        record = {
            "order_id": order.order_id,
            "customer": order.customer_name,
            "items": order.items,
            "total": order.total_amount,
            "type": order.order_type,
            "status": order.status,
            "created_at": order.created_at,
        }
        with open(self.filepath, "a") as f:
            f.write(json.dumps(record) + "\n")
        print(f"    [File] Order #{order.order_id} saved to '{self.
              filepath}' right!")

# ORDER PRICING STRATEGY (LSP + OCP)
# Different order types override only what they need.

class OrderPricing(ABC):
    @abstractmethod
    def final_amount(self, order: Order) -> float:
        """Return the payable amount."""

class RegularOrderPricing(OrderPricing):
    def final_amount(self, order: Order) -> float:
        return order.total_amount # No change

class DiscountedOrderPricing(OrderPricing):
    def __init__(self, discount_pct: float = 10):
        self.discount_pct = discount_pct

    def final_amount(self, order: Order) -> float:
        discount = order.total_amount * self.discount_pct / 100
        print(f"    [Discount] {self.discount_pct}% off -> saving
              Rs.{discount:.2f}")
        return order.total_amount - discount

class PriorityOrderPricing(OrderPricing):
    SURCHARGE = 50 # flat priority fee

```

```

def final_amount(self, order: Order) -> float:
    print(f"    [Priority] Rs.{self.SURCHARGE} surcharge added
          for express delivery")
    return order.total_amount + self.SURCHARGE

# ORDER SERVICE - High-level orchestrator
# DIP: depends only on abstractions injected at runtime
# SRP: only responsibility is to coordinate the flow

class OrderService:
    def __init__(
        self,
        payment: PaymentProcessor,          # abstraction, not
            CreditCardPayment
        notifications: list[NotificationSender],
        storage: OrderStorage,
        pricing: OrderPricing,
    ):
        self._payment = payment
        self._notifications = notifications
        self._storage = storage
        self._pricing = pricing

    def place_order(self, order: Order) -> None:
        print(f"\n{'='*50}")
        print(f"    ORDER #{order.order_id} | {order.order_type} |
              {order.customer_name}")
        print(f"    Items : {'', '.join(order.items)}")
        print(f"{'-'*50}")

        # 1. Calculate final amount
        amount = self._pricing.final_amount(order)

        # 2. Process payment
        success = self._payment.process_payment(amount)
        if not success:
            print("    x Payment failed. Order cancelled.")
            return

        # 3. Update status & save
        order.status = "Confirmed"
        self._storage.save(order)

        # 4. Notify via all configured channels
        for notifier in self._notifications:
            notifier.send(order)

        print(f"    Order #{order.order_id} completed!\n")

```

```

# DEMO - Wiring everything together

if __name__ == "__main__":

    # --- Order 1: Regular order, Credit Card, Email + SMS ---
    o1 = Order(customer_name="Arjun Sharma", items=["Laptop", "
        Mouse"], total_amount=75000, order_type="Regular")
    svc1 = OrderService(
        payment=CreditCardPayment(),
        notifications=[EmailNotification(), SMSNotification()],
        storage=DatabaseStorage(),
        pricing=RegularOrderPricing(),
    )
    svc1.place_order(o1)

    # --- Order 2: Discounted order, UPI, Push notification ---
    o2 = Order(customer_name="Priya Nair", items=["Headphones"],
        total_amount=3000, order_type="Discounted")
    svc2 = OrderService(
        payment=UPIPayment(),
        notifications=[PushNotification()],
        storage=FileStorage("orders.json"),
        pricing=DiscountedOrderPricing(discount_pct=15),
    )
    svc2.place_order(o2)

    # --- Order 3: Priority order, Wallet (sufficient funds) ---
    o3 = Order(customer_name="Ravi Kumar", items=["Keyboard", "
        Monitor"], total_amount=15000, order_type="Priority")
    svc3 = OrderService(
        payment=WalletPayment(balance=20000),
        notifications=[EmailNotification(), PushNotification()],
        storage=DatabaseStorage(),
        pricing=PriorityOrderPricing(),
    )
    svc3.place_order(o3)

    # --- Order 4: Wallet with INSUFFICIENT funds ---
    o4 = Order(customer_name="Neha Singh", items=["Smartwatch"],
        total_amount=12000, order_type="Regular")
    svc4 = OrderService(
        payment=WalletPayment(balance=500),
        notifications=[SMSNotification()],
        storage=FileStorage("orders.json"),
        pricing=RegularOrderPricing(),
    )
    svc4.place_order(o4)

```


Output (Sample)

The program outputs:

- Order details
- Payment status
- Storage confirmation
- Notifications sent

```
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT12> python system.py

ORDER #bf2afe84 | Regular | Arjun Sharma
Items : Laptop, Mouse

[Credit Card] Charged ₹75000.00 - Approved ✓
[DB] INSERT order #bf2afe84 -> orders table ✓
[Email] Order #bf2afe84 confirmed for Arjun Sharma
[SMS] Hi Arjun Sharma! Your order #bf2afe84 is placed.
Order #bf2afe84 completed!

ORDER #73e6d3b | Discounted | Priya Nair
Items : Headphones

[Discount] 15% off -> saving ₹450.00
[UPI] Payment of ₹2550.00 via UPI - Success ✓
[File] Order #73e6d3b saved to 'orders.json' ✓
[Push] Order #73e6d3b placed successfully!
Order #73e6d3b completed!

ORDER #08fa822d | Priority | Ravi Kumar
Items : Keyboard, Monitor

[Priority] ₹50 surcharge added for express delivery
[Wallet] Debited ₹15050.00 | Remaining: ₹4950.00 ✓
[DB] INSERT order #08fa822d -> orders table ✓
[Email] Order #08fa822d confirmed for Ravi Kumar
[Push] Order #08fa822d placed successfully!
Order #08fa822d completed!

ORDER #73e6d3b | Discounted | Priya Nair
Items : Headphones

[Discount] 15% off -> saving ₹450.00
[UPI] Payment of ₹2550.00 via UPI - Success ✓
[File] Order #73e6d3b saved to 'orders.json' ✓
[Push] Order #73e6d3b placed successfully!
Order #73e6d3b completed!

ORDER #08fa822d | Priority | Ravi Kumar
Items : Keyboard, Monitor

[Priority] ₹50 surcharge added for express delivery
[Wallet] Debited ₹15050.00 | Remaining: ₹4950.00 ✓
[DB] INSERT order #08fa822d -> orders table ✓
[Email] Order #08fa822d confirmed for Ravi Kumar
[Push] Order #08fa822d placed successfully!
Order #08fa822d completed!

ORDER #097e18c2 | Regular | Neha Singh
Items : Smartwatch

[Wallet] Insufficient balance X
X Payment failed. Order cancelled.
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT12>
```

Figure 1: Program Output

Learning Outcomes

- Understanding of SOLID principles in real-world applications
- Ability to design modular and scalable systems

- Use of abstraction and polymorphism
- Implementation of dependency injection

Conclusion

The E-commerce Order Processing System successfully demonstrates how SOLID principles improve code maintainability, scalability, and readability. The design allows easy extension of features like new payment methods, storage types, or notification channels without modifying existing code.